**SOFTWARE ENGINEERING LAB TASK 5**

**01-01-2025**

**HU22CSEN0100287**

**SAI GANESH ESWARAPRASAD**

Implement weather modelling using the quadratic solution in stages: hard coding variables keyboard input, read from a file, for a single set of input, multiple sets of inputs. save all versions, debug, fix problems, create a GitHub account**.**

**Aim:**

To model temperature changes over time using the quadratic equation:
**Temperature=a×(time)^2+b×(time)+c**

This program will demonstrate multiple stages of input processing and ensure the solution is debugged and saved to GitHub.

**Steps for Implementation**

**1. Hardcoding Variables**

- Use predefined values for coefficients a, b, and c.
- This stage uses fixed values for the coefficients of the quadratic equation
- This step serves as a proof of concept to test the formula and ensure the program produces correct results before introducing dynamic inputs.

HU22CSEN0100287

Task 3.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

```python
# Hardcoding the coefficients and time
a, b, c = 1, -2, 1  # Coefficients for the quadratic equation
time = 5  # Time in hours
temperature = a * time**2 + b * time + c
print(f"Temperature at time {time} hours: {temperature}")
```

```
Temperature at time 5 hours: 16
```

## 2. Accepting Variables via Keyboard Input

Here, the program dynamically accepts inputs from the user using Python's input() function. This makes the program interactive and flexible.

Task 3.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

```python
# Accepting coefficients and time from the user
a = float(input("Enter coefficient a: "))
b = float(input("Enter coefficient b: "))
c = float(input("Enter coefficient c: "))
time = float(input("Enter time in hours: "))
temperature = a * time**2 + b * time + c
print(f"Temperature at time {time} hours: {temperature}")
```

```
Enter coefficient a: 1
Enter coefficient b: 2
Enter coefficient c: 3
Enter time in hours: 4
Temperature at time 4.0 hours: 27.0
```

Create respective files and upload to drive

| Name | Reason suggested | Owner | Location |
|---|---|---|---|
| multiple_inputs.txt | You uploaded • 11:12 PM | me | My Drive |
| Task 3.ipynb | You modified • 11:11 PM | me | Colab Notebooks |
| Task 2.ipynb | You opened • 11:04 PM | me | Colab Notebooks |
| single_input.txt | You opened • 11:06 PM | me | My Drive |
| inputs.txt | You uploaded • 6:54 PM | me | My Drive |
| input.txt | You created • 6:52 PM | me | My Drive |

## 3. Reading Variables from a File

### (a) Single Set of Inputs

In this step, the coefficients a, b, c, and time are read from a file that contains a single line of data.

Input File (single_input.txt): 1, -2, 1, 5

```
[6] from google.colab import drive
    drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

# Reading a single set of coefficients from a file
with open('single_input.txt', 'r') as file:
    line = file.readline()
    a, b, c, time = map(float, line.split(','))
    temperature = a * time**2 + b * time + c
    print(f"Temperature at time {time} hours: {temperature}")

Temperature at time 5.0 hours: 16.0
```

### (b) Multiple Sets of Inputs

This step handles a file with multiple lines, where each line represents a set of coefficients and time values.

**Input File Example (multiple_inputs.txt):**

H0220100287

```python
# Reading multiple sets of coefficients from a file
with open('multiple_inputs.txt', 'r') as file:
    for line in file:
        a, b, c, time = map(float, line.split(','))
        temperature = a * time**2 + b * time + c
        print(f"Temperature at time {time} hours: {temperature}")
```

```
Temperature at time 5.0 hours: 16.0
Temperature at time 10.0 hours: 197.0
Temperature at time 3.0 hours: 5.0
```

Start coding or generate with AI.

## 4. Debugging and Error Handling

Add checks to prevent common runtime errors such as missing files or incorrect input formats.

**File Not Found Error:**

```python
[11] try:
        with open('multiple_inputs.txt', 'r') as file:
            data = file.read()
    except FileNotFoundError:
        print("Error: The specified file does not exist.")
```

```python
[12] try:
        with open('single_input.txt', 'r') as file:
            data = file.read()
    except FileNotFoundError:
        print("Error: The specified file does not exist.")
```

### Invalid Input Format Error:

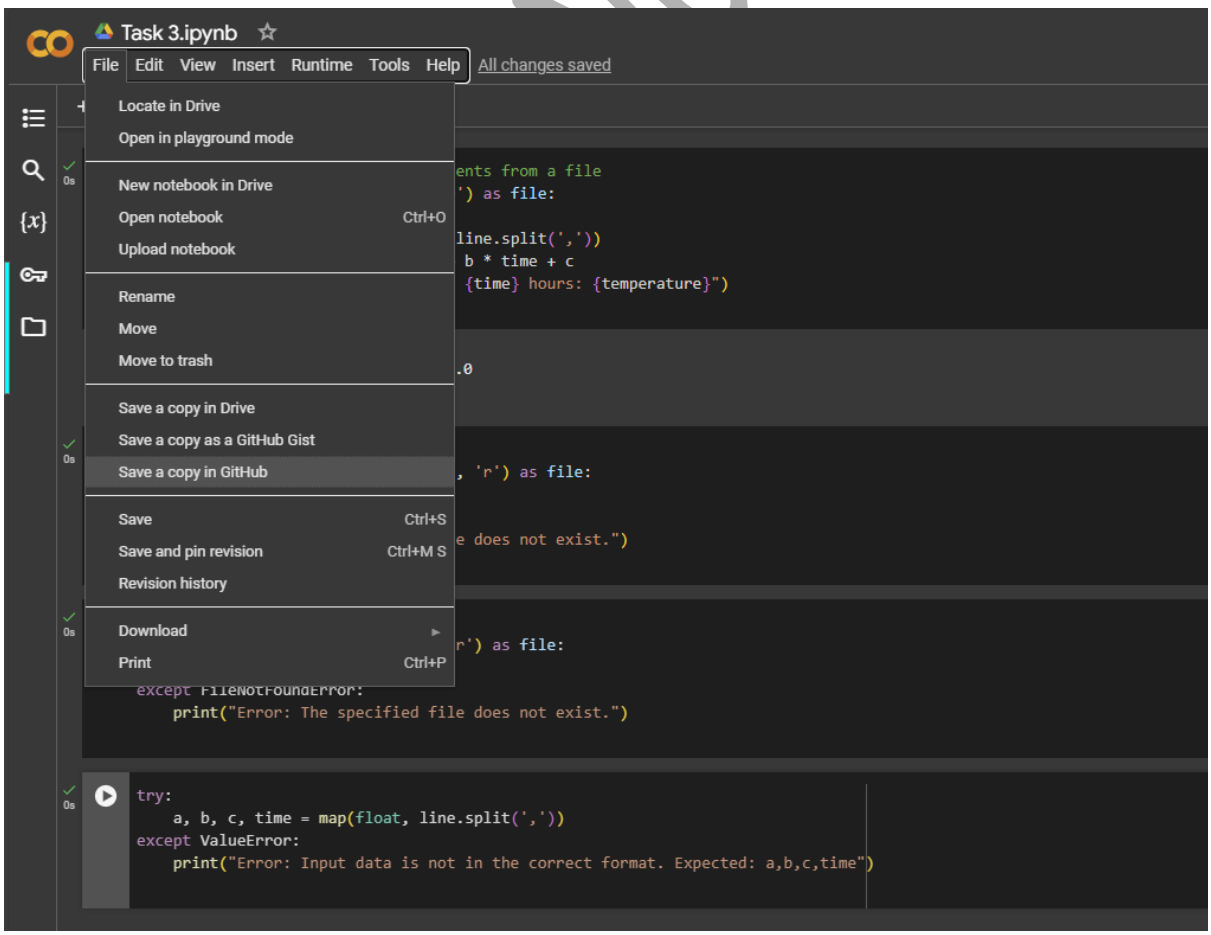
```
try:
    a, b, c, time = map(float, line.split(','))
except ValueError:
    print("Error: Input data is not in the correct format. Expected: a,b,c,time")
```

## 5. Saving Versions

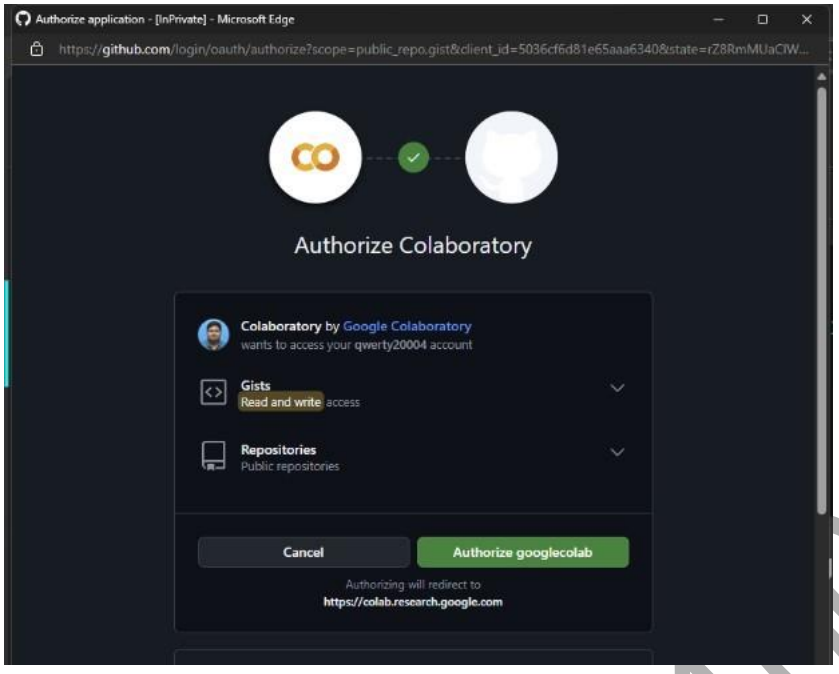
After each stage, save the implementation in a separate code cell or file for record-keeping.

## 6. Pushing the Project to GitHub

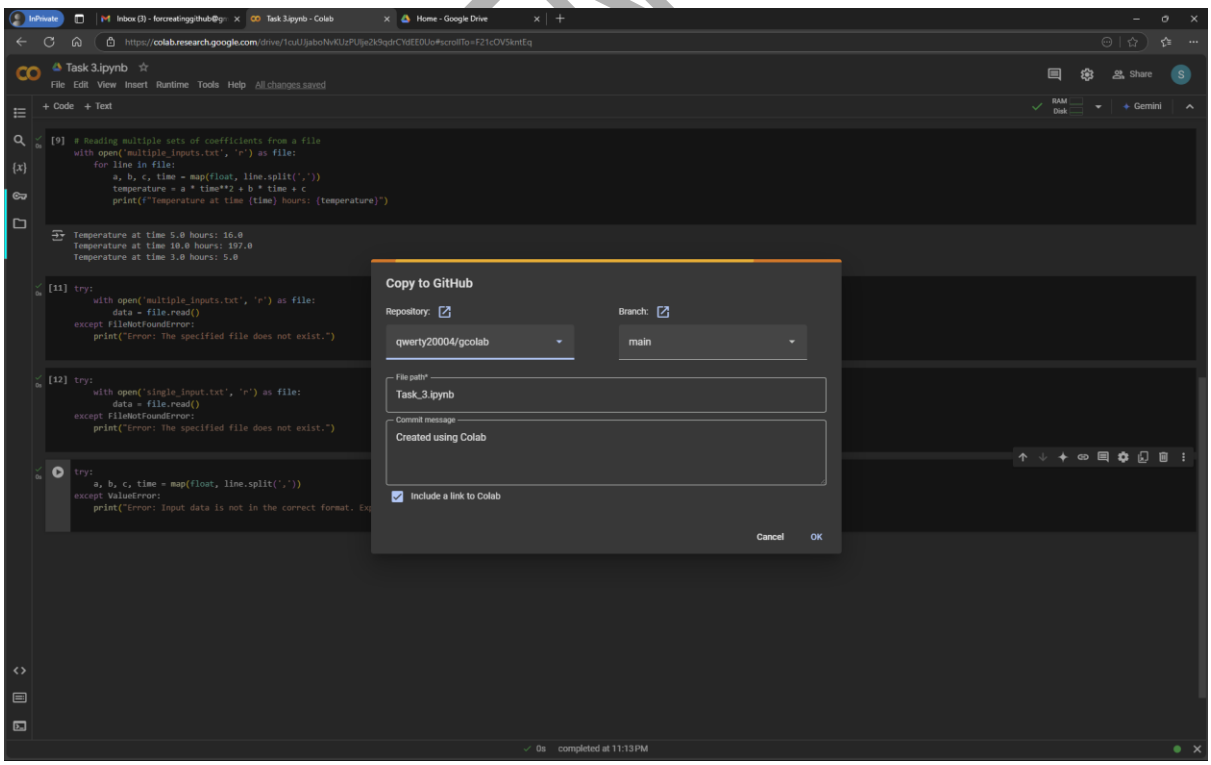
Step 1: Click on file and select option “Save a copy in GitHub”



Step 2: Authorize Collaboratory

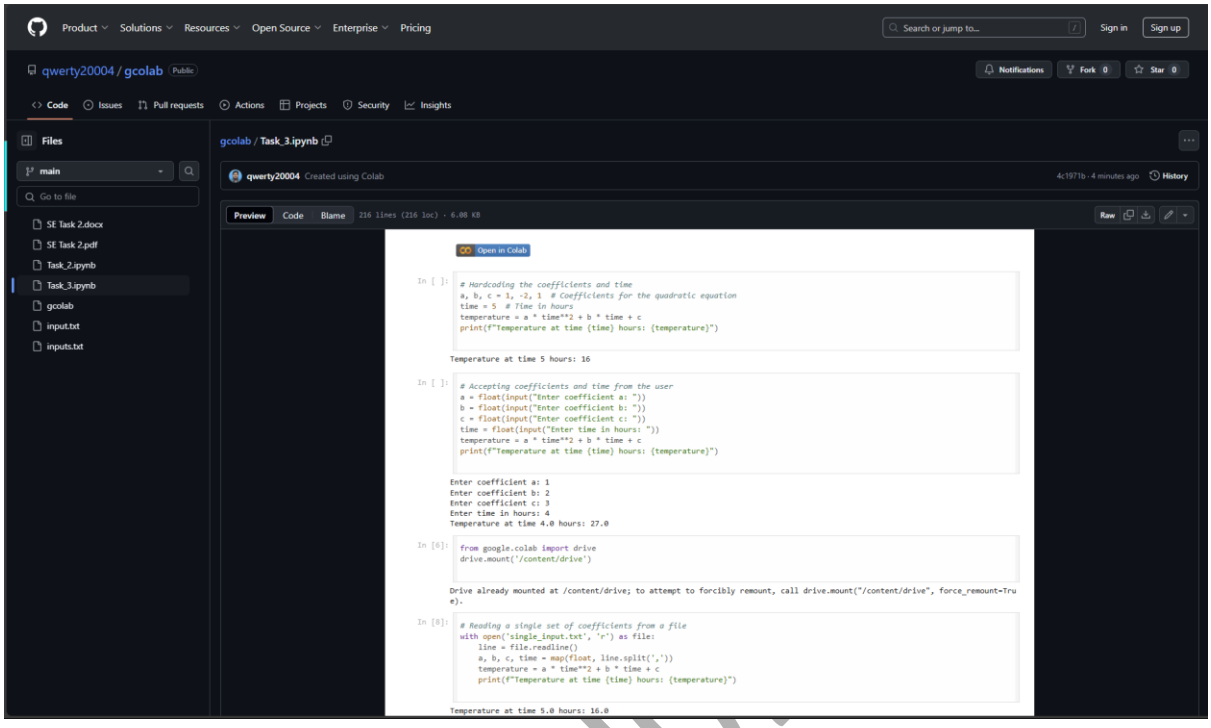



Step 3: Click on 'OK' for copying to GitHub.




N010287

## Step 4: Pushed into GitHub




HU22CSEN0

## Differences between Cohesion and Coupling

| | Cohesion | Coupling |
|---|---|---|
| 1 | Cohesion is the degree to which the elements inside a module belong together. | Coupling is the degree of interdependence between the modules. |
| 2 | A module with high cohesion contains elements that are tightly related to each other and united in their purpose. | Two modules have high coupling (or tight coupling) if they are closely connected and dependent on each other. |
| 3 | A module is said to have low cohesion if it contains unrelated elements. | Modules with low coupling among them work mostly independently of each other. |
| 4 | Highly cohesive modules reflect higher quality of software design | Loose coupling reflects the higher quality of software design |

# Cohesion

- ***cohesion*** refers to the degree to which elements within a module work together to fulfill a single, well-defined purpose.
- High cohesion means that elements are closely related and focused on a single purpose,
- while low cohesion means that elements are loosely related and serve multiple purposes.
- Levels of cohesion!
- Functional!
- Layer!
- Communicational!

# Cohesion

**1.Functional Cohesion**:

1. Functional cohesion occurs when elements of a module (such as functions or methods) are grouped together because they all contribute to a single, well-defined task or functionality.
2. Example: A module that calculates the area of a circle, computes its circumference, and checks if a point lies within the circle exhibits functional cohesion.

**2.Sequential Cohesion**:

1. Sequential cohesion occurs when elements of a module are grouped together because they are executed in a specific sequence, with the output of one element serving as the input to the next.
2. Example: A module that reads data from a file, processes it, and then writes the results to another file exhibits sequential cohesion.

# Cohesion

**3. Communicational Cohesion**:

1. Communicational cohesion occurs when elements of a module are grouped together because they operate on the same data or share the same input/output parameters.
2. Example: A module that performs various operations (such as validation, formatting, and storage) on a user's personal information exhibits communicational cohesion.

**4. Procedural Cohesion**:

1. Procedural cohesion occurs when elements of a module are grouped together because they are all part of a specific procedural or algorithmic sequence.
2. Example: A module that implements a sorting algorithm, including steps for comparing elements, swapping positions, and iterating through the array, exhibits procedural cohesion.

# Cohesion

**5. Temporal Cohesion**:

1. Temporal cohesion occurs when elements of a module are grouped together because they are all executed at the same time, such as during a specific phase or event.
2. Example: A module that initializes system resources, performs setup operations, and starts background tasks during system startup exhibits temporal cohesion.

**6. Coincidental Cohesion**:

1. Coincidental cohesion occurs when elements of a module are grouped together arbitrarily, without any logical relationship between them.
2. Example: A utility module that contains unrelated functions for performing various unrelated tasks, such as file I/O, string manipulation, and mathematical calculations, exhibits coincidental cohesion.

# Coupling

Coupling is a qualitative measure of the degree to which classes are connected to one another.

As classes (and components) become more interdependent, coupling increases.

An important objective in component-level design is to keep coupling as low as is possible.

**Content Coupling**:

1. Definition: Occurs when one component directly modifies data that belongs to another component

Example: A function within Module A directly modifies a variable that is supposed to be private to Module B.

# Coupling

**Common Coupling**:

1. Definition: Multiple components rely on a global variable, potentially leading to unintended consequences when changes are made.
2. Example: Several functions in different modules all use a global configuration variable to determine behavior.

**Data Coupling**:

1. Definition: Operations passing large amounts of data as arguments, increasing interface complexity and making testing and maintenance more difficult.
2. Example: A function passes a complex data structure (e.g., an array or object) to another function for processing.

**Routine Call Coupling**:

1. Definition: One operation invokes another, increasing system connectedness, which is common but can contribute to complexity.
2. Example: Function A calls Function B to perform a specific task, relying on its functionality to achieve a larger goal.

**Inclusion or Import Coupling**:

1. Definition: A component imports content from another component, potentially leading to dependencies between them.
2. Example: Module A includes the header file of Module B, creating a dependency on its contents.

**External Coupling**:

1. Definition: A component communicates or collaborates with external infrastructure components, which should be limited to a small number within a system.
2. Example: An application interacts with the operating system's file system functions to read and write files.